Trying to program like that in CoffeeScript is working against the grain. The objects are dynamic and prototype-based at their core. Work with the language.

Rendering is a function that you just happen to invoke on different objects, some of them products and some of them announcements. The function doesn't need to belong to a *particular* class. You can define a render function anywhere in CoffeeScript. The trouble is, when you can define it anywhere, you're spoiled for choice. If you can define it anywhere, where do you actually define it in practice, and how do you keep related functions together? You want collections of related functions that can be used with objects regardless of what class they were created from—if they were created from a class at all! You want mixins.

5.7.2. Writing a mixin

The simplest way to write a mixin is by taking advantage of the properties as key-value store objects. Define rendering behavior on a plain, old object:

```
htmlRenderer =
  render: ->
    unless @view?
      @view = document.createElement 'div'
      document.body.appendChild @view
      @view.innerHTML = """
      #{@name}
      #{@info}
      """
```

An object created ex nihilo with a render property referring to a function

To add the `render` method to a class, you modify the class prototype:

```
class Donut
  constructor: (name,info) ->
    @name = name
    @info = info
```

Now add the `render` method:

```
Donut::render = htmlRenderer.render
```